

Министерство науки и высшего образования Российской Федерации  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
ИТМО

ITMO University

**Лабораторная работа 4**

**По дисциплине** Администрирование платформ на ОС Linux

**Тема работы** Установка и настройка PostgreSQL, организация резервного копирования и хранение данных в S3-хранилище.

**Обучающийся** Рекун Ирина Данииловна

**Факультет** прикладной информатики

**Группа** K3220

**Направление подготовки** 11.03.02 Инфокоммуникационные технологии и системы связи

**Образовательная программа** Программирование в инфокоммуникационных системах

**Обучающийся**

\_\_\_\_\_  
(дата)

\_\_\_\_\_  
(подпись)

Рекун И.Д.

\_\_\_\_\_  
(Ф.И.О.)

**Руководитель**

\_\_\_\_\_  
(дата)

\_\_\_\_\_  
(подпись)

Береснев А.Д.

\_\_\_\_\_  
(Ф.И.О.)



## ВВЕДЕНИЕ

**Цель работы:** получить практический опыт базовой установки PostgreSQL, настройки памяти и использования ядер CPU, настройки сетевого доступа, управления пользователями и ролями, а также разных видов бэкапа (логический и физический).

**Задачи:** в ходе выполнения лабораторной работы необходимо:

- ❖ Установить и проверить работоспособность системы управления базами данных PostgreSQL
- ❖ Определить основные параметры конфигурации PostgreSQL
- ❖ Создать базу данных и таблицу в ней
- ❖ Заполнить таблицу тестовыми данными
- ❖ Создать нового пользователя и настроить для него права доступа
- ❖ Настроить аутентификацию пользователей в PostgreSQL
- ❖ Проверить локальное и удаленное подключение к базе данных
- ❖ Выполнить резервное копирование базы данных с использованием утилиты `pg_dump`
- ❖ Восстановить базу данных из резервной копии
- ❖ Разработать `bash`-скрипт для автоматизации процесса резервного копирования
- ❖ Загрузить файл резервной копии в S3-совместимое объектное хранилище

На первом этапе лабораторной работы была выполнена проверка доступности пакета PostgreSQL в репозиториях операционной системы с использованием команды `apt policy postgresql` (Рисунок 1).

```
root@rekunid:~# apt policy postgresql
postgresql:
  Installed: (none)
  Candidate: 16+257build1.1
  Version table:
     16+257build1.1 500
        500 http://mirror.selectel.ru/ubuntu noble-updates/main amd64 Packages
     16+257build1 500
```

Рисунок 1 – Проверка доступности пакета PostgreSQL

Далее была выполнена проверка состояния службы PostgreSQL. По результатам видно, что служба успешно загружена и активирована. В журнале также отображается успешный запуск службы без ошибок (Рисунок 2).

```
root@rekunid:~# systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Tue 2026-04-14 15:20:01 UTC; 36s ago
     Main PID: 2928 (code=exited, status=0/SUCCESS)
        CPU: 2ms

Apr 14 15:20:01 rekunid systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Apr 14 15:20:01 rekunid systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
```

Рисунок 2 – Состояние службы PostgreSQL

После этого была проведена проверка запущенных процессов PostgreSQL. В результате выполнения команды были обнаружены основные процессы сервера базы данных, включая главный процесс `postgres`, а также вспомогательные процессы (Рисунок 3).

```
Apr 14 15:20:01 rekunid systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Apr 14 15:20:01 rekunid systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
root@rekunid:~# ps -ef | grep postgres
postgres 3791      1  0 15:20 ?        00:00:00 /usr/lib/postgresql/16/bin/postgres -D /var/lib/postgresql/16/main -c config_file=/etc/postgresql/16/main/postgresql.conf
postgres 3792    3791  0 15:20 ?        00:00:00 postgres: 16/main: checkpointer
postgres 3793    3791  0 15:20 ?        00:00:00 postgres: 16/main: background writer
postgres 3795    3791  0 15:20 ?        00:00:00 postgres: 16/main: walwriter
postgres 3796    3791  0 15:20 ?        00:00:00 postgres: 16/main: autovacuum launcher
postgres 3797    3791  0 15:20 ?        00:00:00 postgres: 16/main: logical replication launcher
root      3874    1044  0 15:21 pts/0    00:00:00 grep --color=auto postgres
root@rekunid:~#
```

Рисунок 3 – Запущенные процессы PostgreSQL

После успешного запуска сервера PostgreSQL была выполнена проверка существующих баз данных с помощью команды `\l`. В результате выполнения команды был получен список стандартных баз данных. Также отображаются параметры кодировки, права доступа (Рисунок 4).

```
postgres=# \l
```

| List of databases |          |          |                 |         |         |            |           |                         |
|-------------------|----------|----------|-----------------|---------|---------|------------|-----------|-------------------------|
| Name              | Owner    | Encoding | Locale Provider | Collate | Ctype   | ICU Locale | ICU Rules | Access privileges       |
| postgres          | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |            |           | =c/postgres +           |
| template0         | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |            |           | postgres=Ctc/postgres + |
| template1         | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |            |           | =c/postgres +           |
| (3 rows)          |          |          |                 |         |         |            |           | postgres=Ctc/postgres   |

Рисунок 4 – Список баз данных PostgreSQL

Далее была выполнена проверка основных конфигурационных файлов PostgreSQL с помощью SQL-команд. В результате были получены пути к файлу основной конфигурации, файлу аутентификации клиентов и каталогу хранения данных. Это позволяет определить, где находятся ключевые настройки сервера и где физически хранятся базы данных. Полученные значения подтверждают стандартное расположение файлов в системе Linux (Рисунок 5).

```
root@rekunid:~# sudo -u postgres psql
psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# SHOW config_file;
          config_file
-----
/etcd/postgresql/16/main/postgresql.conf
(1 row)

postgres=# SHOW hba_file;
          hba_file
-----
/etcd/postgresql/16/main/pg_hba.conf
(1 row)

postgres=# SHOW data_directory;
      data_directory
-----
/var/lib/postgresql/16/main
(1 row)
```

Рисунок 5 – Пути к конфигурационным файлам и каталогу данных

После определения расположения конфигурационных файлов были получены основные параметры системы: объем оперативной памяти — 3.8 ГБ и количество ядер CPU — 2. На основе этих данных были настроены параметры PostgreSQL. Значение `shared_buffers` установлено равным 1 ГБ ( $\approx 25\%$  ОЗУ), `work_mem` — 2 МБ, `maintenance_work_mem` — 64 МБ. Также заданы параметры параллелизма: `max_worker_processes` = 3, `max_parallel_workers_per_gather` = 1 и `max_parallel_workers` = 2 (Рисунок 6).

```
max_worker_processes = 3
max_parallel_workers = 2
max_parallel_workers_per_gather = 1

shared_buffers = 1GB
work_mem = 2MB
maintenance_work_mem = 64MB
```

Help Write Out Where Is

Рисунок 6 – Параметры конфигурации PostgreSQL

После настройки параметров была выполнена их проверка с использованием команд SHOW. В результате было подтверждено, что значения `shared_buffers`, `work_mem`, `maintenance_work_mem` и `max_worker_processes` успешно применены и соответствуют заданным ранее настройкам (Рисунок 7).

```
postgres=# SHOW shared_buffers;
SHOW shared_buffers;
shared_buffers
-----
1GB
(1 row)

work_mem
-----
2MB
(1 row)

maintenance_work_mem
-----
64MB
(1 row)

max_worker_processes
-----
3
(1 row)

postgres=# SHOW shared_buffers;
SHOW shared_buffers;
shared_buffers
-----
1GB
(1 row)

postgres=# SHOW shared_buffers;
SHOW shared_buffers;
shared_buffers
-----
1GB
(1 row)
```

Рисунок 7 – Проверка параметров PostgreSQL

Далее была создана новая база данных `debian_kernels`, после чего выполнено подключение к ней. Внутри базы данных была создана таблица `debian_versions`, содержащая информацию о версиях Debian, их кодовых именах и версиях ядра. После создания таблицы в нее были добавлены тестовые данные, и с помощью команды `SELECT` была выполнена проверка их корректного сохранения (Рисунок 8).

```
psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# CREATE DATABASE debian_kernels;
CREATE DATABASE
postgres=# \c debian_kernels
You are now connected to database "debian_kernels" as user "postgres".
debian_kernels=# CREATE TABLE debian_versions (
    id SERIAL PRIMARY KEY,
    debian_version VARCHAR(20) NOT NULL,
    codename VARCHAR(50),
    kernel_version VARCHAR(20) NOT NULL
);
CREATE TABLE
debian_kernels=# INSERT INTO debian_versions (debian_version, codename, kernel_version) VALUES
('10', 'Buster', '4.19'),
('11', 'Bullseye', '5.10'),
('12', 'Bookworm', '6.1'),
('13', 'Trixie', '6.12');
INSERT 0 4
debian_kernels=# SELECT * FROM debian_versions;
 id | debian_version | codename | kernel_version
-----+-----+-----+-----
  1 | 10             | Buster  | 4.19
  2 | 11             | Bullseye | 5.10
  3 | 12             | Bookworm | 6.1
  4 | 13             | Trixie  | 6.12
(4 rows)

debian_kernels=#
```

Рисунок 8 – Создание базы данных, таблицы и добавление данных

На следующем этапе был создан пользователь с возможностью входа в систему. Данному пользователю были выданы все необходимые права на базу данных и схему `public`. Проверка списка ролей показала, что пользователь успешно создан и обладает заданными правами (рисунок 9).

```
postgres=# CREATE ROLE app_user WITH LOGIN PASSWORD '12345678';
CREATE ROLE
postgres=# GRANT ALL PRIVILEGES ON DATABASE debian_kernels TO app_user;
GRANT
postgres=# \c debian_kernels
You are now connected to database "debian_kernels" as user "postgres".
debian_kernels=# GRANT ALL ON SCHEMA public TO app_user;
GRANT
debian_kernels=# \du

                                List of roles
Role name |                               Attributes
-----+-----
 app_user |
 postgres | Superuser, Create role, Create DB, Replication, Bypass RLS
```

Рисунок 9 – Создание пользователя и назначение прав доступа

Для обеспечения удаленного подключения к серверу PostgreSQL был изменен параметр `listen_addresses`, который был установлен в значение `'*'`. Это позволяет серверу принимать подключения с любых сетевых интерфейсов (Рисунок 10).

```
# - Connection Settings -
listen_addresses = '*'          # what IP address(es) to listen on;
                                # comma-separated list of addresses
```

Рисунок 10 – Настройка сетевого доступа PostgreSQL

Далее была выполнена проверка открытого порта 5432. В результате было подтверждено, что PostgreSQL прослушивает соединения на всех адресах (0.0.0.0) (Рисунок 11).

```
root@rekunid:~# ss -tulpen | grep 5432
tcp    LISTEN 0      200      0.0.0.0:5432      0.0.0.0:*        users:((("postgres",pid=1194,fd=6)) uid:106 ino:10292
ice/system-postgresql.slice/postgresql@16-main.service <->
tcp    LISTEN 0      200      [::]:5432        [::]:*          users:((("postgres",pid=1194,fd=7)) uid:106 ino:10293
lice/system-postgresql.slice/postgresql@16-main.service v6only:1 <->
```

Рисунок 11 – Проверка порта

После этого было выполнено подключение к базе данных с удаленного хоста с использованием пользователя `app_user`. (Рисунок 12).

```
root@rekunid:~# psql -h 178.72.166.217 -U app_user -d debian_kernels -W
Password:
psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.
```

Рисунок 12 – Подключение к базе данных PostgreSQL с удаленного хоста

Далее было выполнено создание резервной копии базы данных с использованием утилиты `pg_dump`. В результате выполнения команды был сформирован файл `debian_kernels_backup.sql`. (Рисунок 13).

```
root@rekunid:~# PGPASSWORD=12345678 pg_dump -U app_user -h 127.0.0.1 -d debian_kernels > debian_kernels_backup.sql
root@rekunid:~# ls -lh debian_kernels_backup.sql
-rw-r--r-- 1 root root 2.8K Apr 15 13:34 debian_kernels_backup.sql
root@rekunid:~# sudo -u postgres psql
psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))
Type "help" for help.
```

Рисунок 13 – Создание резервной копии базы данных

Далее происходит восстановление базы данных из ранее созданного резервного файла в новую базу данных `debian_kernels_test`. В процессе выполнения команды были получены предупреждения, связанные с ограничениями прав доступа и ролей, однако основные операции, включая создание таблицы и загрузку данных, были выполнены успешно (Рисунок 14).



```

root@rekunid:~# PGPASSWORD=12345678 psql -U app_user -h 127.0.0.1 -d debian_kernels_test < debian_kernels_backup.sql
SET
SET
SET
SET
SET
set_config
-----
(1 row)

SET
SET
SET
SET
SET
SET
CREATE TABLE
ERROR: must be able to SET ROLE "postgres"
CREATE SEQUENCE
ERROR: must be able to SET ROLE "postgres"
ALTER SEQUENCE
ALTER TABLE
COPY 4
setval
-----
      4
(1 row)

ALTER TABLE
GRANT
GRANT
GRANT

```

Рисунок 14 – Восстановление базы данных из резервной копии

После этого был создан скрипт автоматического резервного копирования базы данных backup.sh. Скрипту были выданы права на выполнение, после чего он был успешно запущен. (Рисунок 15).

```

root@rekunid:~# nano backup.sh
root@rekunid:~# chmod +x backup.sh
root@rekunid:~# ./backup.sh
Backup успешно создан: /root/debian_kernels_2026-04-15.sql

```

Рисунок 15 – Выполнение скрипта резервного копирования

На завершающем этапе резервная копия базы данных была загружена в объектное хранилище Selectel с использованием S3-совместимого API. Файл был успешно передан в бакет irinalogs (рисунок 15).

```

root@rekunid:~# aws s3 cp /root/debian_kernels_2026-04-15.sql s3://irinalogs/2026/
upload: ./debian_kernels_2026-04-15.sql to s3://irinalogs/2026/debian_kernels_2026-04-15.sql

```

Рисунок 16 – Загрузка резервной копии в S3-хранилище

Резервная копия базы данных была успешно загружена. Файл debian\_kernels\_2026-04-15.sql размещён в бакете irinalogs (рисунок 17).

s3://irinalogs/2026/ 

2 объекта

Имя объекта

| <input type="checkbox"/> | Наименование                                                                          |
|--------------------------|---------------------------------------------------------------------------------------|
| <input type="checkbox"/> |  04/ |
| <input type="checkbox"/> | debian_kernels_2026-04-15.sql                                                         |

Рисунок 17 – Копия в бакете

## **ЗАКЛЮЧЕНИЕ**

В ходе работы был установлен и настроен PostgreSQL, создана база данных и пользователь, а также настроено удалённое подключение. Были выполнены резервное копирование и восстановление базы данных. Дополнительно был создан скрипт для автоматического бэкапа и настроена загрузка файлов в S3-хранилище. В результате были получены практические навыки работы с PostgreSQL и резервным копированием данных.

